

C++ & Qt 과제 보고서(Day2)

학번: 2026406076

이름: 정창규

제출일: 2026-09-04

목차

1. 과제 개요
2. 개발 환경 및 프로젝트 구성
3. 과제 1 - 로봇 컨트롤 패널
 - 3.1 UI 구성
 - 3.2 방향 제어 및 상태 표시
 - 3.3 속도 조절
 - 3.4 실행 결과
4. 과제 2 - 카페 주문 대기열
 - 4.1 UI 구성
 - 4.2 Vector를 이용한 Queue 구현
 - 4.3 주문 처리
 - 4.4 실행 결과

1. 과제 개요

이번 과제에서는 C++과 Qt Creator를 이용하여 두 개의 GUI 프로그램을 제작하였다.

과제 1에서는 QPushButton, QLabel, QSlider와 Signal / Slot을 이용하여 간단한 로봇 컨트롤 패널을 구현하였다. 전진, 후진, 좌회전, 우회전, 정지 기능을 버튼으로 조작하고 현재 상태와 속도를 화면에 표시하도록 하였다.

과제 2에서는 C++의 std::vector를 사용하여 Queue의 FIFO 구조를 구현하였다. 주제는 카페 주문 대기열로 정하고, 주문이 들어온 순서대로 처리되도록 제작하였다.

2. 개발 환경 및 프로젝트 구성

개발 환경은 다음과 같다.

- 운영체제: Ubuntu
- 개발 도구: Qt Creator
- 프로그래밍 언어: C++
- GUI: Qt Widgets
- Build System: CMake

프로젝트는 다음과 같은 구조로 구성하였다.

Project

```
├── CMakeLists.txt
├── include
│   └── mainwindow.h
├── resources
├── src
│   ├── main.cpp
│   └── mainwindow.cpp
└── ui
    └── mainwindow.ui
```

mainwindow.ui에서 화면을 구성하고, mainwindow.cpp에서 버튼과 슬라이더의 동작을 구현하였다.

3. 과제 1 - 로봇 컨트롤 패널

3.1 UI 구성

로봇의 이동 방향을 조작하기 위해 전진, 후진, 좌회전, 우회전, 정지 버튼을 배치하였다.

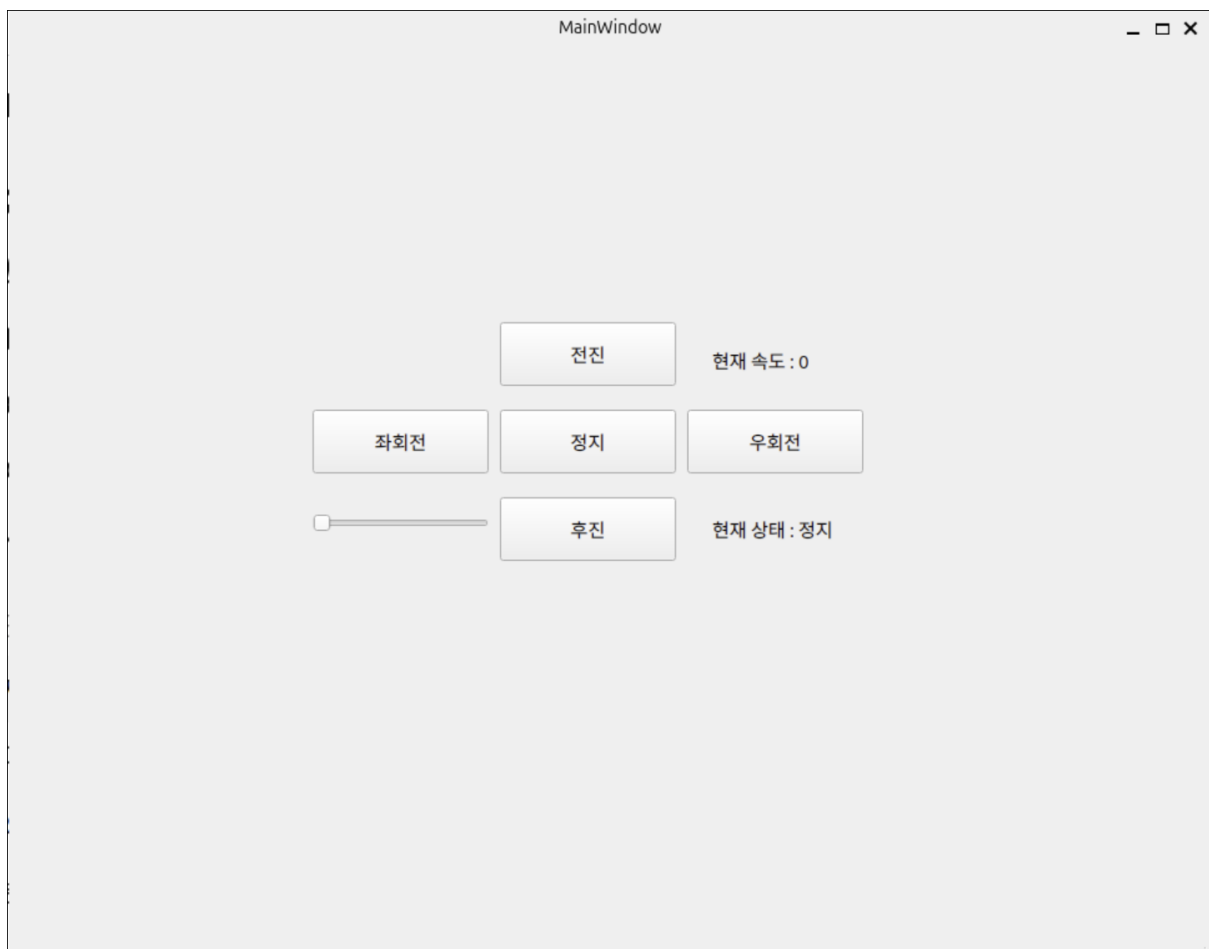
또한 QLabel을 이용해 현재 로봇의 상태를 표시하고, QSlider와 별도의 QLabel을 이용해 현재 속도를 표시하도록 구성하였다.

프로그램 실행 시 초기 상태는 다음과 같다.

현재 상태 : 정지

현재 속도 : 0

그림 1. 로봇 컨트롤 패널 초기 실행 화면



3.2 방향 제어 및 상태 표시

버튼의 clicked() Signal을 각 Slot 함수와 연결하여 버튼 입력에 따라 상태가 변경되도록 하였다.

전진 버튼은 다음과 같이 구현하였다.

```
void MainWindow::on_forward_btn_clicked()
{
    ui->status_label->setText("현재 상태 : 전진");
}
```

전진 버튼을 클릭하면 on_forward_btn_clicked()가 실행되고 QLabel의 내용이 현재 상태 : 전진으로 변경된다.

후진, 좌회전, 우회전, 정지도 동일한 방식으로 구현하였다.

3.3 속도 조절

속도 조절에는 QSlider의 valueChanged(int) Signal을 사용하였다.

```
void MainWindow::on_speed_slider_valueChanged(int value)
{
    ui->speed_label->setText(
        "현재 속도 : " + QString::number(value)
    );
}
```

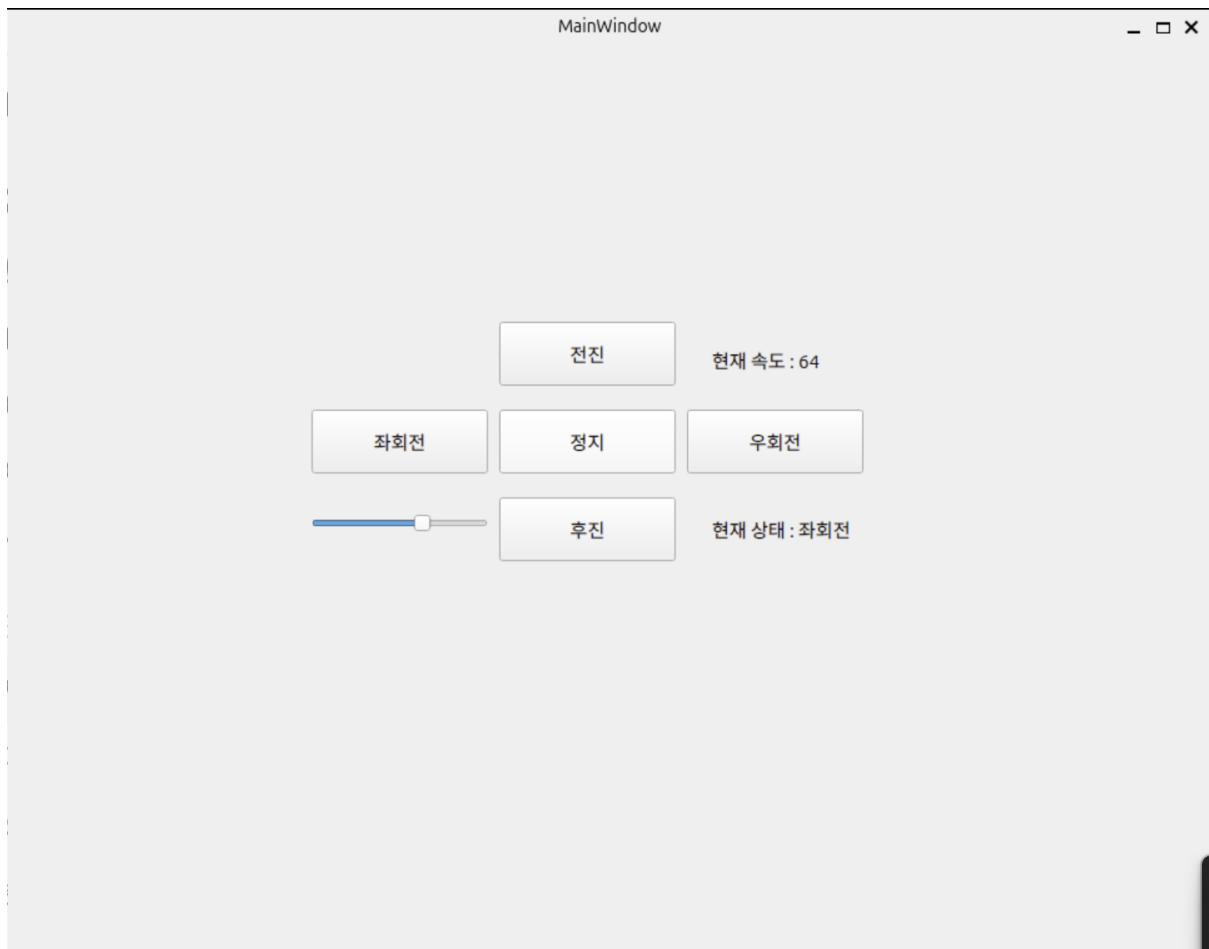
슬라이더가 움직일 때 전달되는 value를 문자열로 변환하여 현재 속도를 화면에 표시하였다.

3.4 실행 결과

전진, 후진, 좌회전, 우회전, 정지 버튼을 각각 눌렀을 때 상태가 정상적으로 변경되는 것을 확인하였다.

또한 속도 슬라이더를 움직였을 때 현재 속도 값이 슬라이더의 위치에 맞게 변경되는 것을 확인하였다.

그림 2. 방향 및 속도 변경 실행 화면



4. 과제 2 - 카페 주문 대기열

4.1 UI 구성

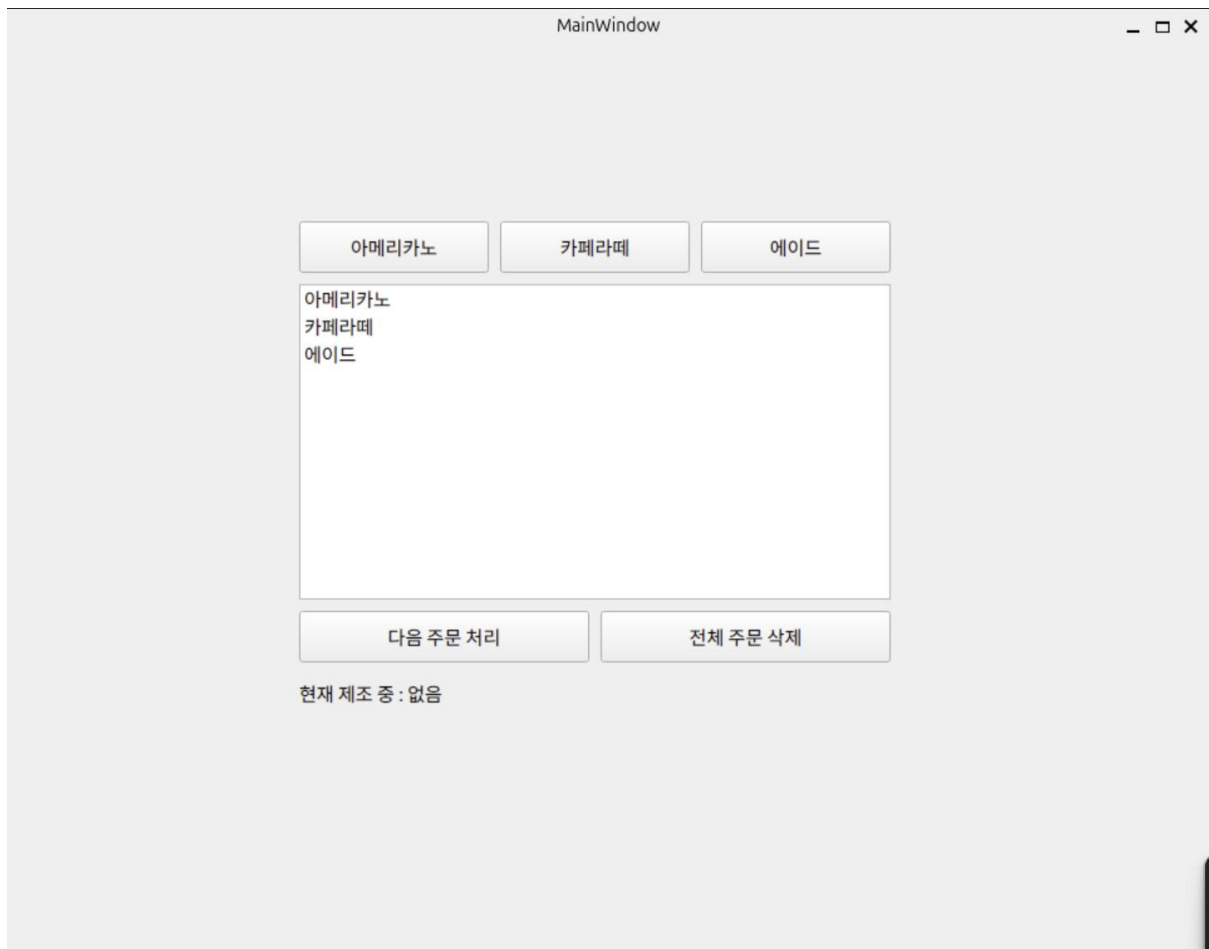
Queue의 FIFO 구조를 확인할 수 있도록 카페 주문 대기열 프로그램을 제작하였다.

메뉴는 다음과 같이 구성하였다.

- 아메리카노
- 카페라떼
- 에이드

주문 목록을 보여주기 위한 QListWidget, 다음 주문을 처리하는 버튼, 전체 주문을 삭제하는 버튼, 현재 제조 중인 메뉴를 표시하는 QLabel을 배치하였다.

그림 3. 카페 주문 대기열 주문 추가 화면



4.2 Vector를 이용한 Queue 구현

주문 데이터를 저장하기 위해 C++의 `std::vector`를 사용하였다.

```
std::vector<QString> orderQueue;
```

메뉴 버튼을 누르면 `push_back()`을 이용하여 새로운 주문을 Vector의 마지막에 추가하였다.

```
void MainWindow::on_americano_btn_clicked()
```

```
{  
    orderQueue.push_back("아메리카노");  
    ui->order_list->addItem("아메리카노");  
}
```

이와 같이 주문 데이터와 화면의 대기 목록을 동시에 추가하였다.

4.3 주문 처리

Queue는 먼저 들어온 데이터가 먼저 나오는 FIFO 구조이므로 Vector의 첫 번째 데이터를 처리하도록 구현하였다.

```
QString order = orderQueue.front();
```

```
ui->current_order_label->setText(  
    "현재 제조 중 : " + order  
);
```

```
orderQueue.erase(orderQueue.begin());
```

```
delete ui->order_list->takeItem(0);
```

front()를 이용하여 가장 먼저 들어온 주문을 가져오고, erase(orderQueue.begin())을 이용하여 처리한 주문을 Vector에서 제거하였다.

예를 들어 아메리카노 → 카페라떼 → 에이드 순서로 주문하면 아메리카노가 가장 먼저 처리된다.

4.4 실행 결과

아메리카노, 카페라떼, 에이드 순서로 주문을 추가하였을 때 주문 목록에도 같은 순서로 표시되었다.

다음 주문 처리 버튼을 누르면 가장 먼저 들어온 아메리카노가 목록에서 제거되고,

현재 제조 중 : 아메리카노로 표시되는 것을 확인하였다.

이후 남은 대기 목록에는 카페라떼와 에이드가 표시되어 FIFO 방식으로 동작하는 것을 확인하였다.

그림 4. 다음 주문 처리 실행 화면

